

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: MLA03 COURSE CODE: Reinforcement learning for Environment and Agent

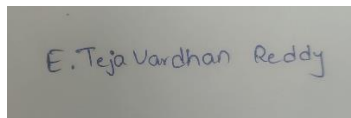
COURSE OUTCOME COVERED

CO1: *Analyze reinforcement learning frameworks and Markov Decision Process concepts to model decision-making problems. (BL2, BL3)*

Assessment Tool Weightage: 40%

Code of Conduct:

***I E TEJA VARDHAN REDDY (192425297)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*



Signature of the student:

Annexure A

Descriptive Experiment Exam

Duration: 2hrs

Experiment 1: Installation of Reinforcement Learning Development Environment

Aim

To install and configure Anaconda Navigator, Python, TensorFlow, Keras, Gymnasium (OpenAI Gym), NumPy, Matplotlib, and other required libraries, and verify the environment by executing a simple Reinforcement Learning program.

Algorithm

1. Install Anaconda Navigator.
2. Create a new Python environment.

3. Install the required libraries:
 - NumPy
 - Matplotlib
 - Gymnasium
 - TensorFlow
 - Keras
4. Import the installed libraries.
5. Create a FrozenLake environment.
6. Reset the environment.
7. Display the initial state.
8. Verify that the environment is working correctly.

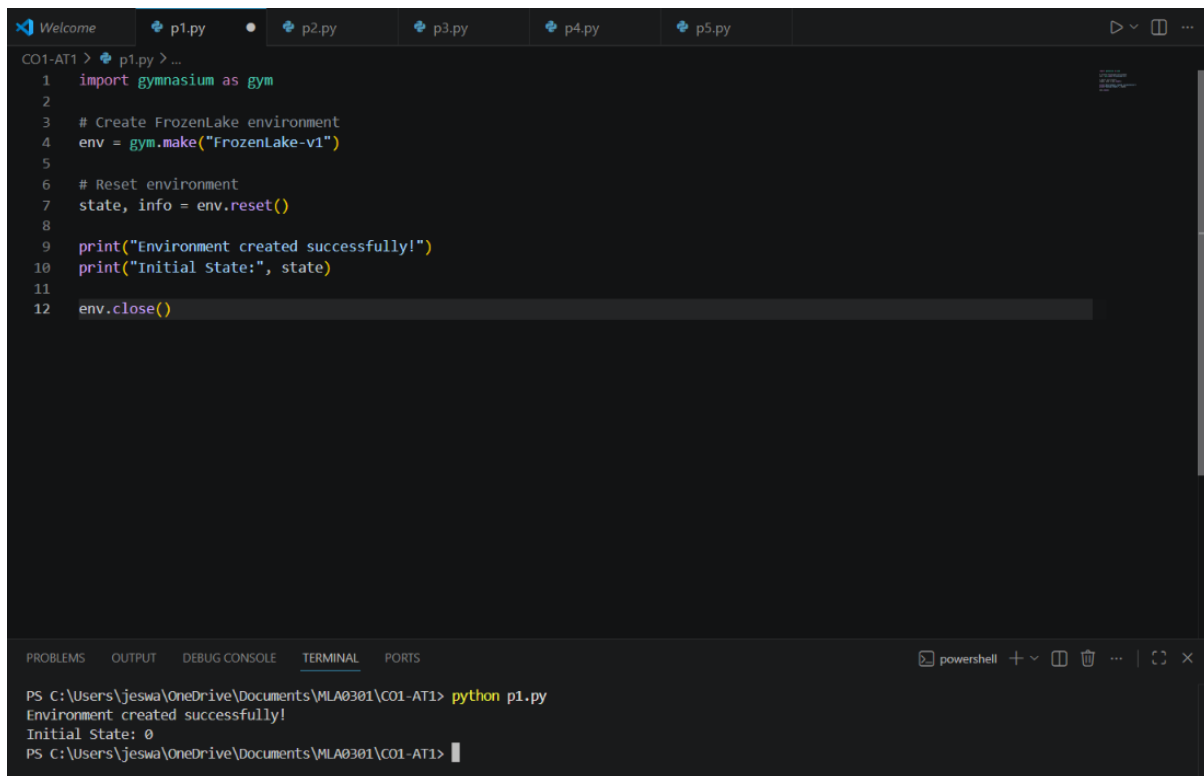
Python Program

```
import gymnasium as gym
# Create FrozenLake environment
env = gym.make("FrozenLake-v1")
# Reset environment
state, info = env.reset()
print("Environment created successfully!")
print("Initial State:", state)
env.close()
```

Input

No user input is required.

Output



The screenshot shows a Visual Studio Code window with a file explorer at the top displaying five Python files: p1.py, p2.py, p3.py, p4.py, and p5.py. The main editor area shows the content of p1.py, which is a Python script for creating and initializing a FrozenLake environment using the gymnasium library. The script includes comments and print statements to verify the environment's creation and state. The bottom panel shows the 'TERMINAL' tab with the command 'python p1.py' executed, resulting in the output: 'Environment created successfully!' and 'Initial State: 0'.

```
CO1-AT1 > p1.py > ...
1 import gymnasium as gym
2
3 # Create FrozenLake environment
4 env = gym.make("FrozenLake-v1")
5
6 # Reset environment
7 state, info = env.reset()
8
9 print("Environment created successfully!")
10 print("Initial State:", state)
11
12 env.close()
```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS powershell + - [] [X] [Y] [Z]

PS C:\Users\jeswa\OneDrive\Documents\MLA0301\CO1-AT1> python p1.py
Environment created successfully!
Initial State: 0
PS C:\Users\jeswa\OneDrive\Documents\MLA0301\CO1-AT1>

Result

The Reinforcement Learning development environment was successfully installed and verified by creating and initializing the FrozenLake environment.

Experiment 2: Familiarization with OpenAI Gym/Gymnasium Environments

Aim

To create, initialize, and interact with standard Reinforcement Learning environments such as FrozenLake using Gymnasium to understand the concepts of states, actions, rewards, and episodes.

Algorithm

1. Import the Gymnasium library.
2. Create the FrozenLake environment.
3. Reset the environment to obtain the initial state.
4. Select a random action.
5. Execute the action using the `step()` function.
6. Observe the next state, reward, and termination status.

7. Display the obtained results.
8. Close the environment.

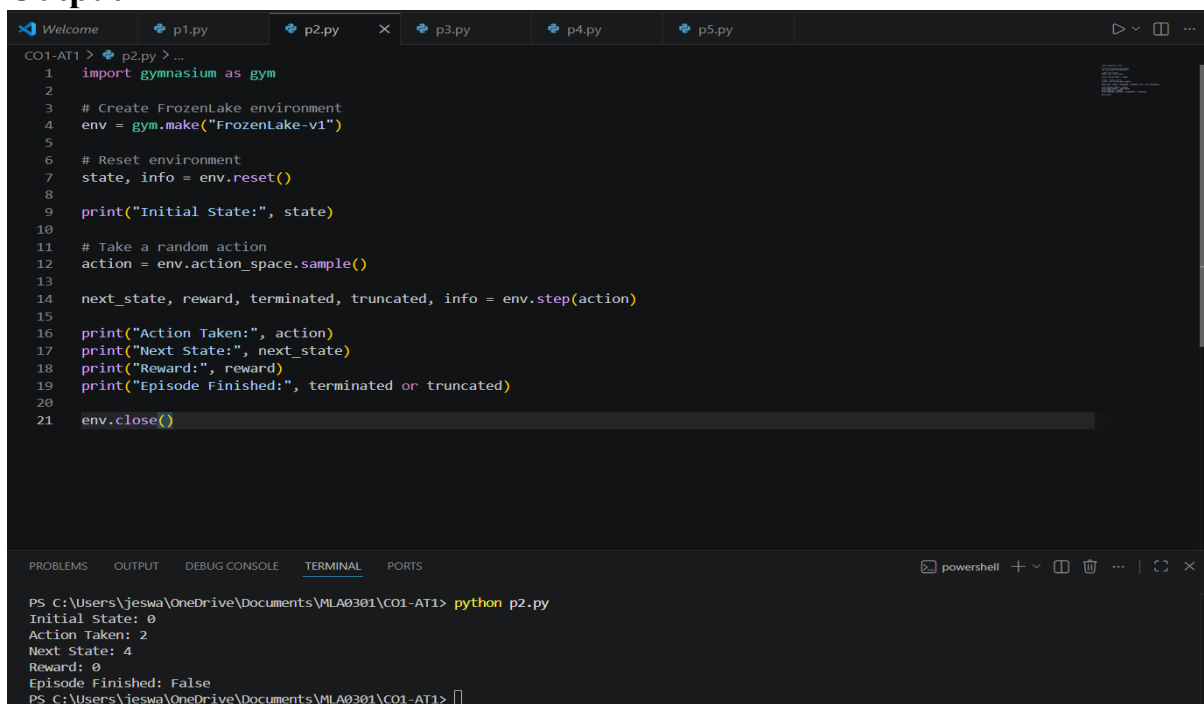
Python Program

```
import gymnasium as gym
# Create FrozenLake environment
env = gym.make("FrozenLake-v1")
# Reset environment
state, info = env.reset()
print("Initial State:", state)
# Take a random action
action = env.action_space.sample()
next_state, reward, terminated, truncated, info = env.step(action)
print("Action Taken:", action)
print("Next State:", next_state)
print("Reward:", reward)
print("Episode Finished:", terminated or truncated)
env.close()
```

Input

No user input is required.

Output



The screenshot shows a Visual Studio Code editor window with a dark theme. The top bar shows the file explorer with several Python files (p1.py, p2.py, p3.py, p4.py, p5.py). The main editor area displays a Python script for the FrozenLake environment. The script imports gymnasium, creates the environment, resets it, takes a random action, and prints the state, action, reward, and episode status. The bottom panel shows the terminal output of the script, which matches the print statements in the code.

```
CO1-AT1 > p2.py > ...
1 import gymnasium as gym
2
3 # Create FrozenLake environment
4 env = gym.make("FrozenLake-v1")
5
6 # Reset environment
7 state, info = env.reset()
8
9 print("Initial State:", state)
10
11 # Take a random action
12 action = env.action_space.sample()
13
14 next_state, reward, terminated, truncated, info = env.step(action)
15
16 print("Action Taken:", action)
17 print("Next State:", next_state)
18 print("Reward:", reward)
19 print("Episode Finished:", terminated or truncated)
20
21 env.close()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\jeswa\OneDrive\Documents\MLA0301\CO1-AT1> python p2.py
Initial State: 0
Action Taken: 2
Next State: 4
Reward: 0
Episode Finished: False
PS C:\Users\jeswa\OneDrive\Documents\MLA0301\CO1-AT1>
```

Result

The Gymnasium FrozenLake environment was successfully created, and the agent interacted with the environment by performing a random action.

Experiment 3: Implementation of the Reinforcement Learning Framework

Aim

To implement the basic Reinforcement Learning framework by designing an agent that interacts with an environment through states, actions, rewards, and policies using Python.

Algorithm

1. Create the FrozenLake environment.
2. Reset the environment.
3. Repeat for five steps:
 - Select a random action.
 - Execute the action.
 - Receive the next state and reward.
 - Display the interaction details.
4. Stop if the episode ends.

Python Program

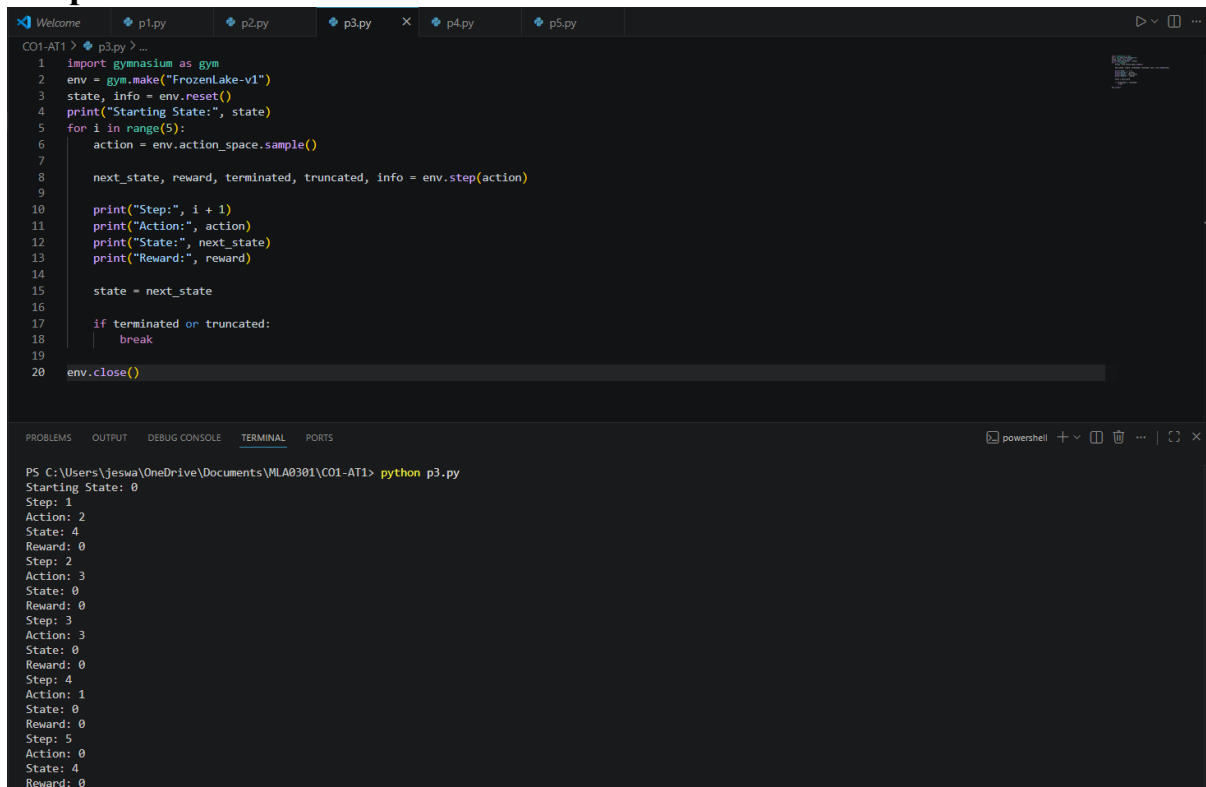
```
import gymnasium as gym
env = gym.make("FrozenLake-v1")
state, info = env.reset()
print("Starting State:", state)
for i in range(5):
    action = env.action_space.sample()
    next_state, reward, terminated, truncated, info = env.step(action)
    print("Step:", i + 1)
    print("Action:", action)
    print("State:", next_state)
    print("Reward:", reward)
    state = next_state
    if terminated or truncated:
```

```
        break
env.close()
```

Input

No user input is required.

Output



```
CO1-AT1 > p3.py > ...
1 import gymnasium as gym
2 env = gym.make("FrozenLake-v1")
3 state, info = env.reset()
4 print("Starting State:", state)
5 for i in range(5):
6     action = env.action_space.sample()
7
8     next_state, reward, terminated, truncated, info = env.step(action)
9
10    print("Step:", i + 1)
11    print("Action:", action)
12    print("State:", next_state)
13    print("Reward:", reward)
14
15    state = next_state
16
17    if terminated or truncated:
18        break
19
20 env.close()
```

```
PS C:\Users\jeswa\OneDrive\Documents\MLA0301\CO1-AT1> python p3.py
Starting State: 0
Step: 1
Action: 2
State: 4
Reward: 0
Step: 2
Action: 3
State: 0
Reward: 0
Step: 3
Action: 3
State: 0
Reward: 0
Step: 4
Action: 1
State: 0
Reward: 0
Step: 5
Action: 0
State: 4
Reward: 0
```

Result

The Reinforcement Learning framework was successfully implemented, demonstrating the interaction between the agent and the environment through states, actions, and rewards.

Experiment 4: Simulation of a Markov Decision Process (MDP)

Aim

To develop a simple Markov Decision Process model and simulate state transitions, reward functions, and policy execution for understanding sequential decision-making problems.

Algorithm

1. Define states and actions.
2. Define state transition probabilities.
3. Define rewards.

4. Choose an action.
5. Display the next state and reward.
6. Repeat for all states.

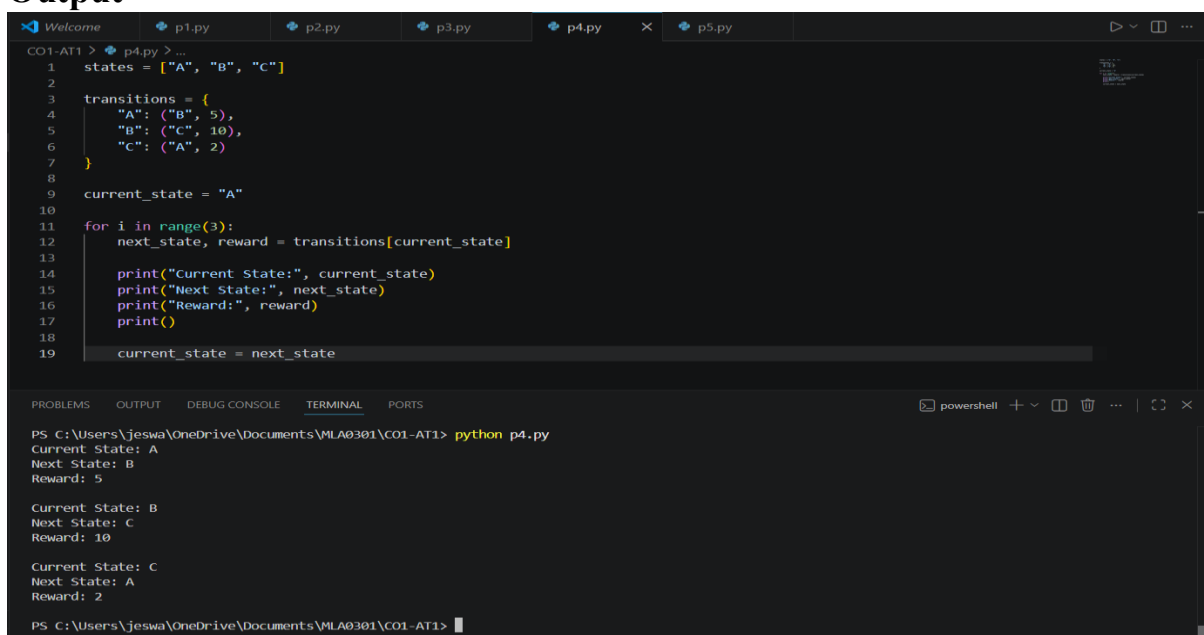
Python Program

```
states = ["A", "B", "C"]
transitions = {
    "A": ("B", 5),
    "B": ("C", 10),
    "C": ("A", 2)
}
current_state = "A"
for i in range(3):
    next_state, reward = transitions[current_state]
    print("Current State:", current_state)
    print("Next State:", next_state)
    print("Reward:", reward)
    print()
    current_state = next_state
```

Input

No user input is required.

Output



The screenshot shows a Visual Studio Code editor with a Python file named p4.py. The code defines a list of states ["A", "B", "C"] and a dictionary of transitions. It then iterates through the states, printing the current state, next state, and reward for each transition. The terminal output shows the execution of the script, displaying the current state, next state, and reward for each transition.

```
1 states = ["A", "B", "C"]
2
3 transitions = {
4     "A": ("B", 5),
5     "B": ("C", 10),
6     "C": ("A", 2)
7 }
8
9 current_state = "A"
10
11 for i in range(3):
12     next_state, reward = transitions[current_state]
13
14     print("Current State:", current_state)
15     print("Next State:", next_state)
16     print("Reward:", reward)
17     print()
18
19     current_state = next_state
```

Terminal Output:

```
PS C:\Users\jeswa\OneDrive\Documents\MLA0301\CO1-AT1> python p4.py
Current State: A
Next State: B
Reward: 5

Current State: B
Next State: C
Reward: 10

Current State: C
Next State: A
Reward: 2

PS C:\Users\jeswa\OneDrive\Documents\MLA0301\CO1-AT1>
```

Result

A simple Markov Decision Process was successfully simulated using state transitions and reward functions.

Experiment 5: Bellman Equation Demonstration

Aim

To implement Bellman Expectation and Bellman Optimality Equations for calculating state-value functions and analyze the convergence of value estimation.

Algorithm

1. Define rewards for each state.
2. Initialize state values to zero.
3. Apply the Bellman equation iteratively.
4. Update the value of each state.
5. Display the updated state values.

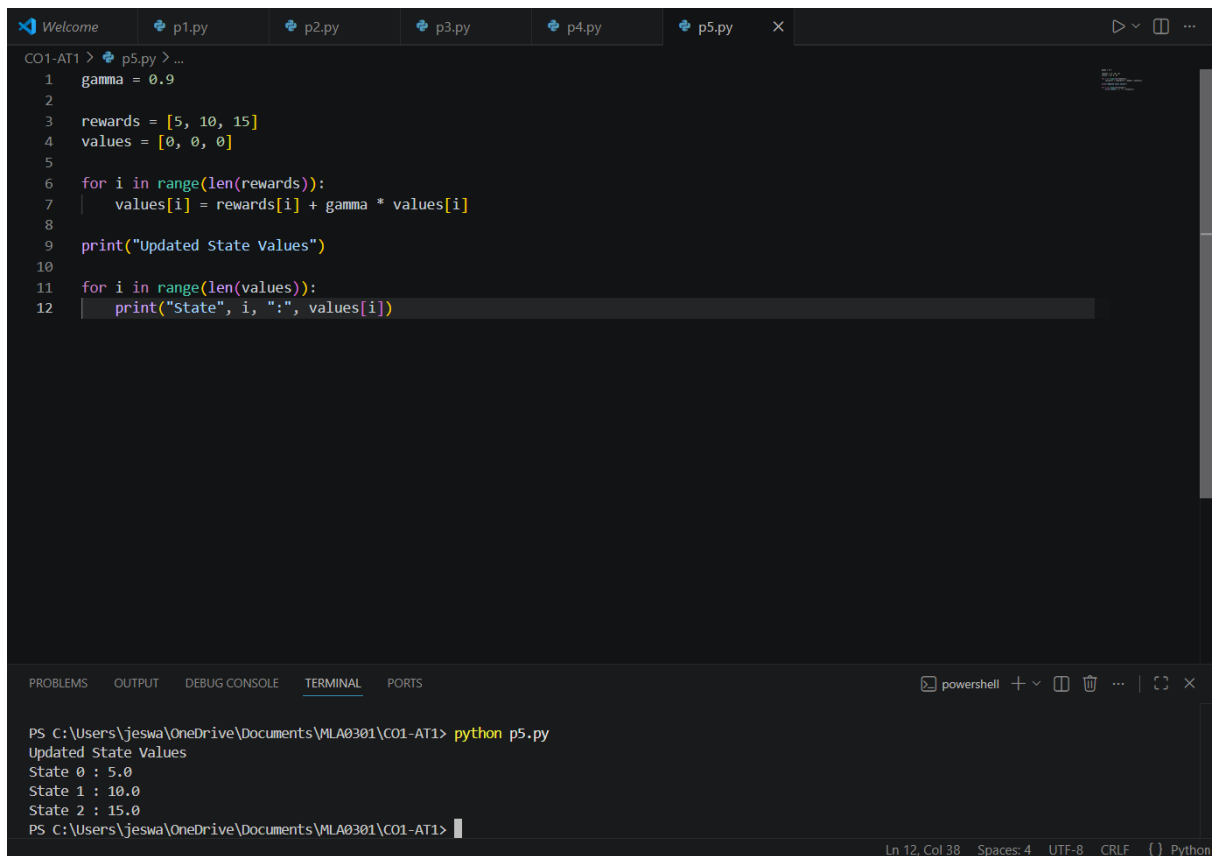
Python Program

```
gamma = 0.9
rewards = [5, 10, 15]
values = [0, 0, 0]
for i in range(len(rewards)):
    values[i] = rewards[i] + gamma * values[i]
print("Updated State Values")
for i in range(len(values)):
    print("State", i, ":", values[i])
```

Input

No user input is required.

Output



The image shows a Visual Studio Code editor window with a dark theme. The top bar displays several open files: 'Welcome', 'p1.py', 'p2.py', 'p3.py', 'p4.py', and 'p5.py'. The active file is 'p5.py', which contains the following Python code:

```
1 gamma = 0.9
2
3 rewards = [5, 10, 15]
4 values = [0, 0, 0]
5
6 for i in range(len(rewards)):
7     values[i] = rewards[i] + gamma * values[i]
8
9 print("Updated State Values")
10
11 for i in range(len(values)):
12     print("State", i, ":", values[i])
```

The bottom panel of the editor shows the 'TERMINAL' tab, which contains the output of running the script. The command prompt shows the file path and the command used to run the script:

```
PS C:\Users\jeswa\OneDrive\Documents\MLA0301\CO1-AT1> python p5.py
```

The output of the script is as follows:

```
Updated State Values
State 0 : 5.0
State 1 : 10.0
State 2 : 15.0
PS C:\Users\jeswa\OneDrive\Documents\MLA0301\CO1-AT1>
```

The status bar at the bottom right indicates the current cursor position (Ln 12, Col 38), the number of spaces (4), the encoding (UTF-8), the line ending (CRLF), and the file type (Python).

Result

The Bellman Equation was successfully implemented to estimate the value of each state.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand